Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224.
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

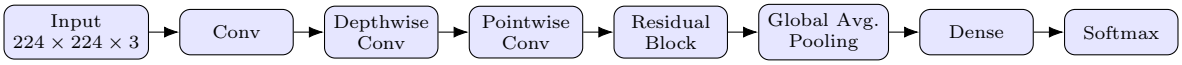
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Expected Analysis

Students should compare the initialization strategies using:

Plot 1: Training Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Training Loss
- Plot one curve for each initialization method.

Plot 2: Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Plot one curve for each initialization method.

Students should identify which initialization gives faster and more stable convergence.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data.

Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Expected Plots

Plot 3: Training and Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Plot separate curves for training and validation accuracy.

Students should identify the generalization gap.

Plot 4: Training and Validation Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Loss
- Plot training and validation loss.

Students should identify whether validation loss begins to increase while training loss continues to decrease.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.
For a mini-batch

$$x_1, x_2, \ldots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

$$\boxed{\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]}$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Plot 5: With vs. Without Batch Normalization

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Compare two curves: With BN and Without BN.

8. Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

Plot 6: Training Loss vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Training Loss
- One curve for each optimizer.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- One curve for each optimizer.

Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time
SGD	3.0794	23.37%	5	42.87 s
Momentum	0.4708	89.13%	5	42.32 s
RMSProp	0.0464	89.81%	5	42.35 s
Adam	0.0435	90.49%	5	42.81 s

9. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	$10^{-4}, 10^{-5}$
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Plot 8: Learning Rate vs. Validation Accuracy

- X-axis: Learning Rate
- Y-axis: Validation Accuracy (%)

Plot 9: Batch Size vs. Validation Accuracy

- X-axis: Batch Size
- Y-axis: Validation Accuracy (%)

Plot 10: Dropout Rate vs. Validation Accuracy

- X-axis: Dropout Rate
- Y-axis: Validation Accuracy (%)

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

Plot 11: Feature Extraction vs. Fine-Tuning

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Two curves: Feature Extraction and Fine-Tuning.

Plot 12: Training and Validation Loss

Students should compare the loss curves before and after fine-tuning.

Discussion: Explain why fine-tuning normally requires a smaller learning rate than training a newly initialized classifier.

11. K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.

Use 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5}\sum_{i=1}^{5} A_i$$

and

$$SD = \sqrt{\frac{1}{5}\sum_{i=1}^{5}(A_i - \bar{A})^2}.$$

Report the result as

$$\boxed{\bar{A} \pm SD}.$$

The independent test set must not be used during hyperparameter selection.

Configuration	F1	F2	F3	F4	F5	Mean ± SD
C1	92.70%	91.00%	91.34%	89.47%	88.61%	90.62 ± 1.44%
C2	92.19%	90.49%	91.51%	92.02%	88.95%	91.03 ± 1.20%
C3	92.36%	90.66%	89.98%	91.85%	89.29%	90.83 ± 1.14%
C4	79.97%	78.95%	75.38%	78.10%	77.04%	77.89 ± 1.58%

Plot 13: 5-Fold Cross-Validation Accuracy

- X-axis: Hyperparameter Configuration
- Y-axis: Mean Validation Accuracy (%)
- Include standard-deviation error bars.

Students should discuss both mean performance and variability.

12. Final Model Evaluation

After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.
2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

Report:

Metric	Value
Mean CV Accuracy	91.03%
CV Standard Deviation	1.20%
Test Accuracy	89.07%
Precision	89.50%
Recall	89.01%
F1-score	89.00%
Training Time	42.45 s
Number of Parameters	2,426,725

Plot 14: Confusion Matrix

Students should identify:

- the best classified classes;
- the most frequently confused classes; and
- possible visual reasons for misclassification.

Optional Plot 15: Misclassified Images

Display representative incorrectly classified images and provide a brief explanation for each.

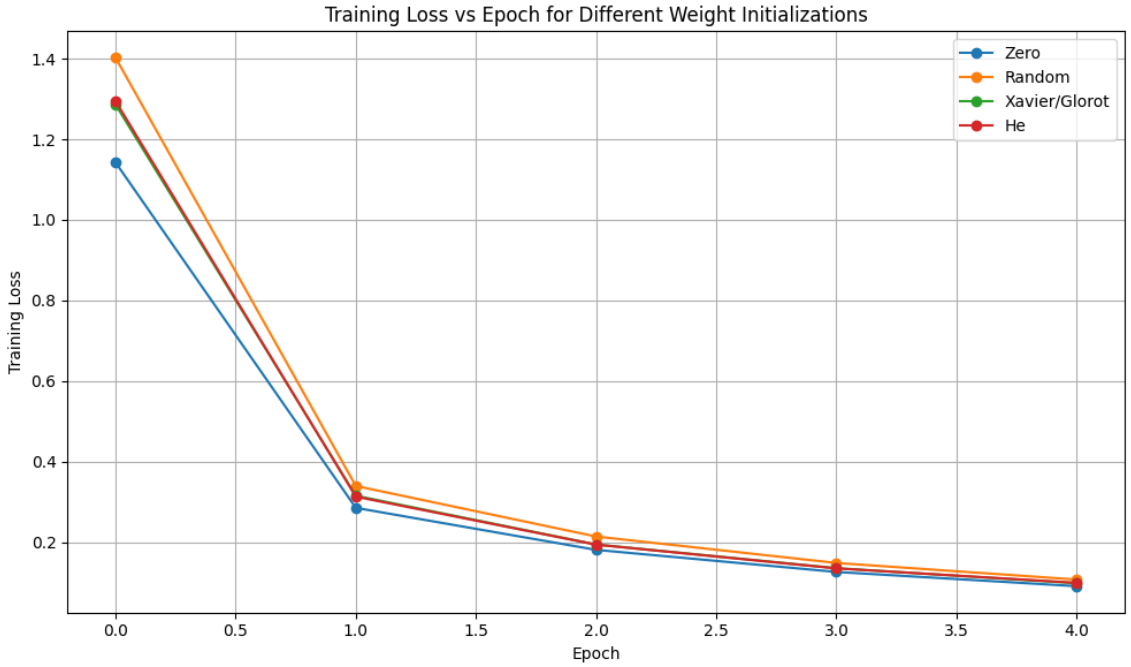
13. Overall Results

Configuration	CV Accuracy	SD	Test Accuracy	Training Time
Baseline (Feature Extraction)	90.62%	1.44%	–	41.74 s
Best Initialization (He)	–	–	–	41.91 s
Best Regularization (Batch Norm.)	–	–	–	42.66 s
Best Optimizer (Adam)	–	–	–	42.81 s
Best Hyperparameters (Dropout 0.25)	91.03%	1.20%	89.07%	42.45 s
Fine-Tuned Model	77.89%	1.58%	–	46.17 s

Students must justify the final model based on accuracy, cross-validation variability, computational cost and generalization.

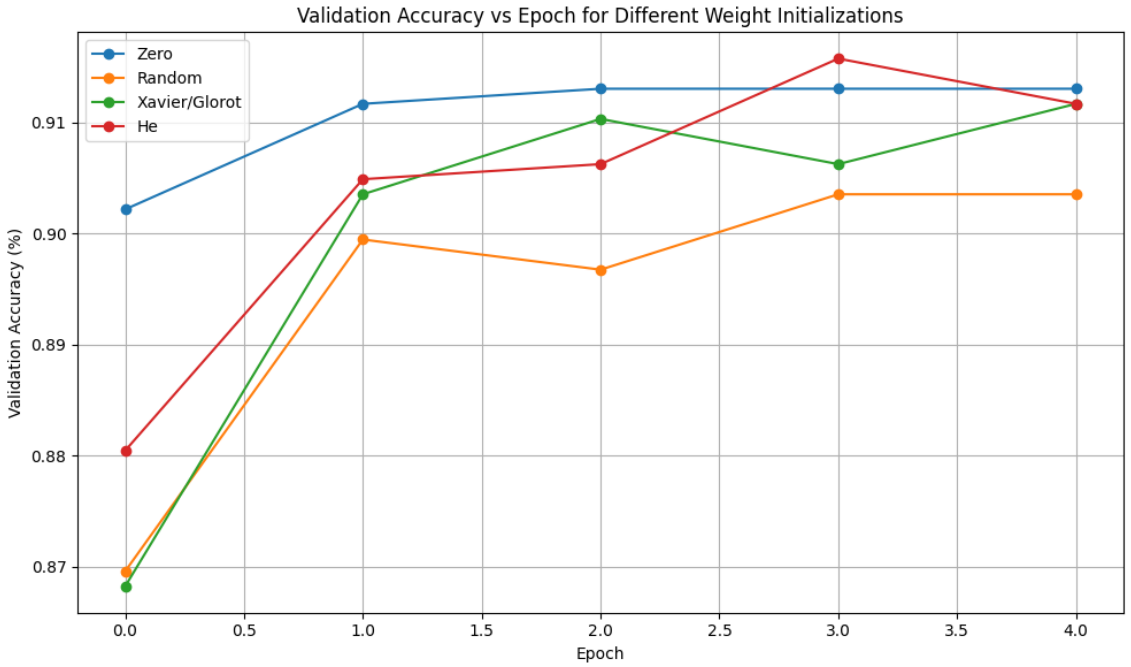
14. Required Inference for Plots

Plot 1: Training Loss vs Epoch for Different Weight Initializations



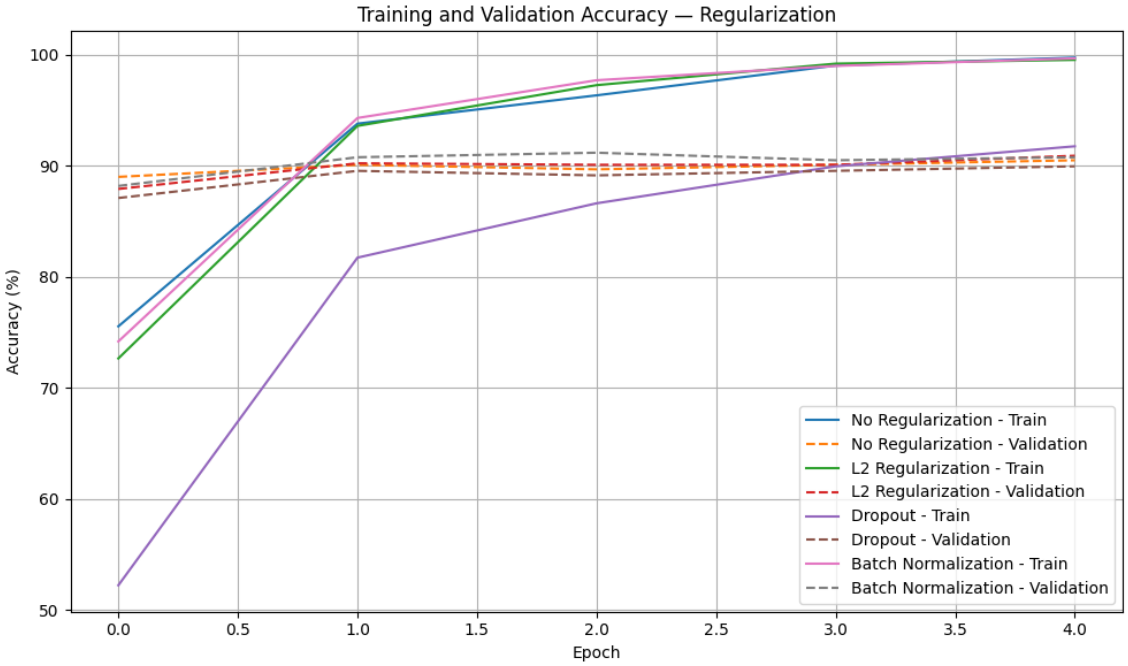
The plot compares the training loss obtained using Zero, Random, Xavier/Glorot, and He initialization. All methods show a decreasing loss trend, indicating successful learning, while He initialization provided the best overall performance. This shows that appropriate weight initialization can improve convergence and model training.

Plot 2: Validation Accuracy vs Epoch for Different Weight Initializations



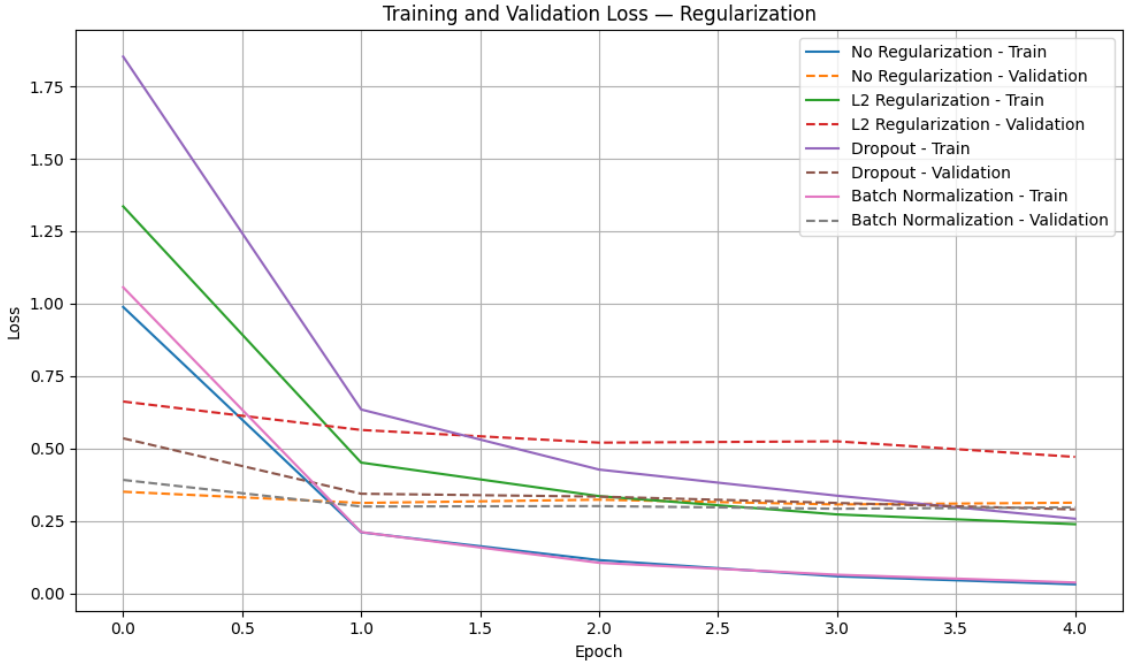
The validation accuracy generally increased during training for all initialization methods. He initialization achieved the best validation performance among the tested methods. The result indicates that initialization affects the learning dynamics and generalization capability of the network.

Plot 3: Training and Validation Accuracy for Regularization Methods



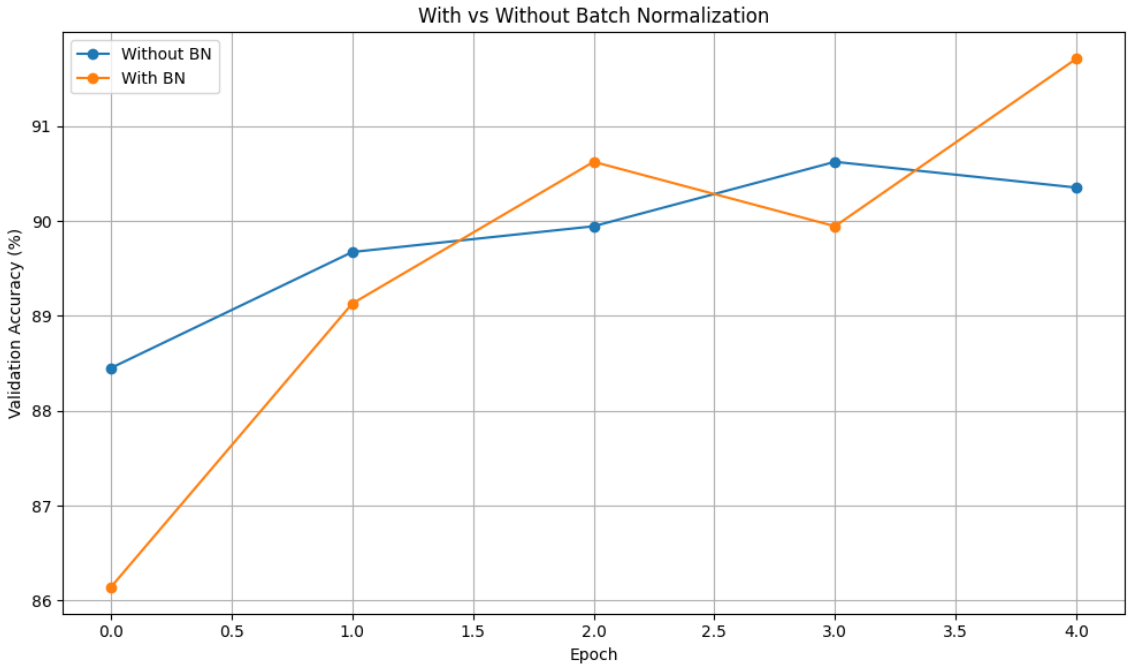
The plot compares training and validation accuracy for No Regularization, L2 Regularization, Dropout, and Batch Normalization. Batch Normalization achieved the highest validation accuracy of approximately 91.17%. Regularization techniques help improve generalization by reducing the gap between training and validation performance.

Plot 4: Training and Validation Loss for Regularization Methods



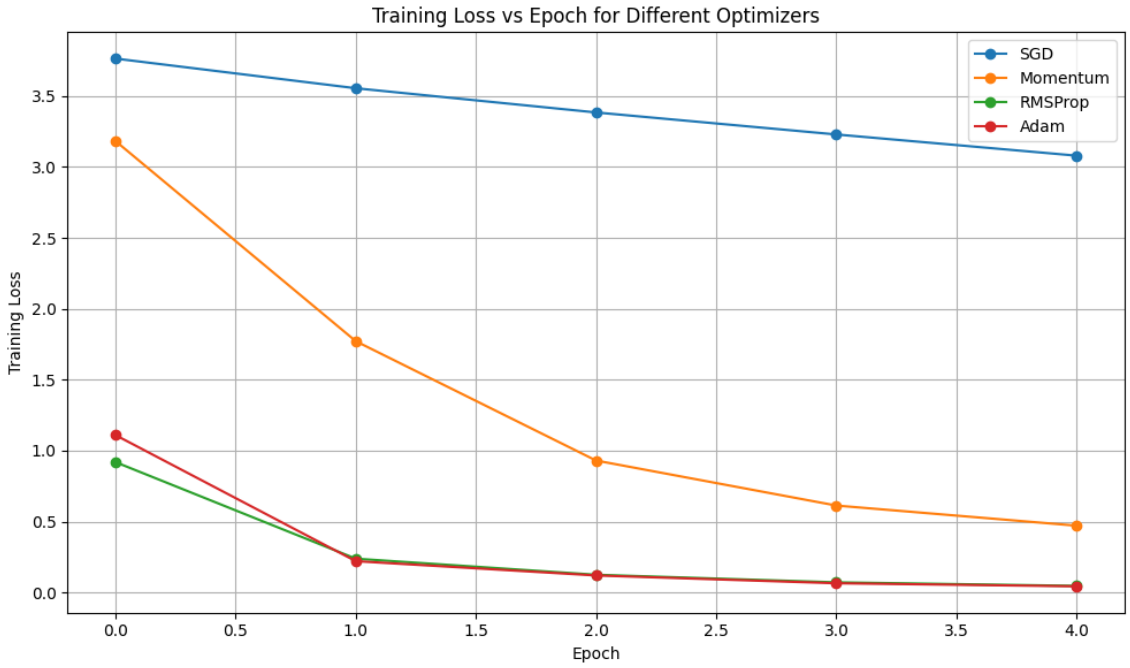
Training loss decreased for all regularization methods, while validation loss showed comparatively smaller changes. The unregularized model achieved very low training loss, indicating possible overfitting. Regularization helps control overfitting by improving the model's ability to generalize.

Plot 5: With vs Without Batch Normalization



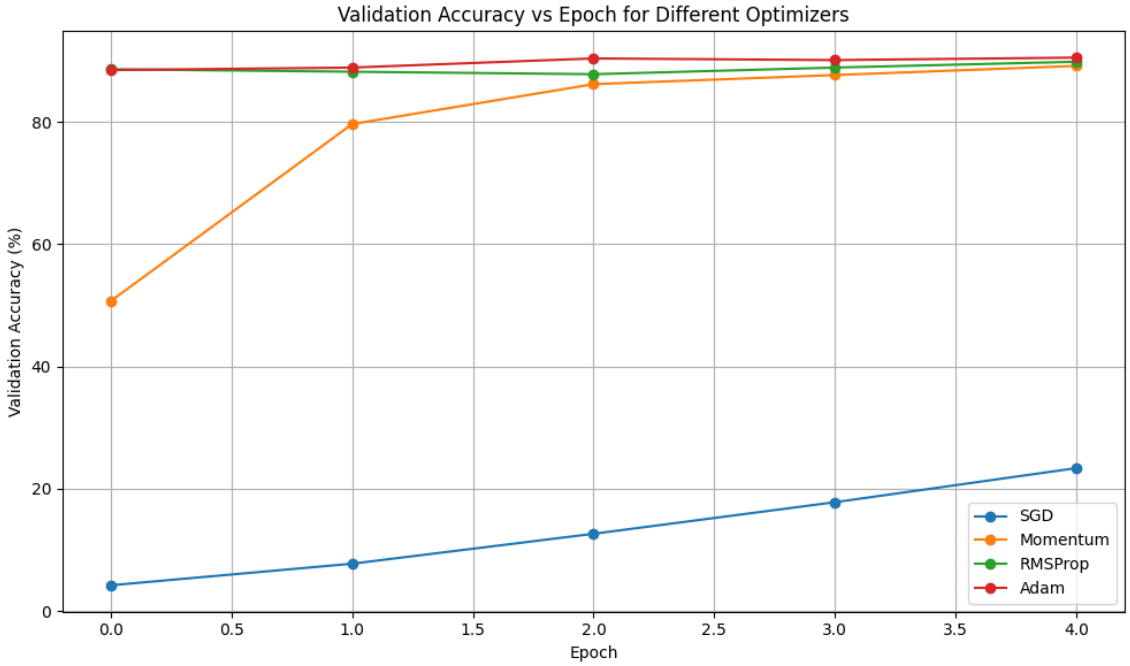
The model with Batch Normalization achieved better validation accuracy, reaching approximately 91.71%, compared with the model without Batch Normalization. Batch Normalization helps stabilize the learning process and can improve convergence and generalization.

Plot 6: Training Loss vs Epoch for Different Optimizers



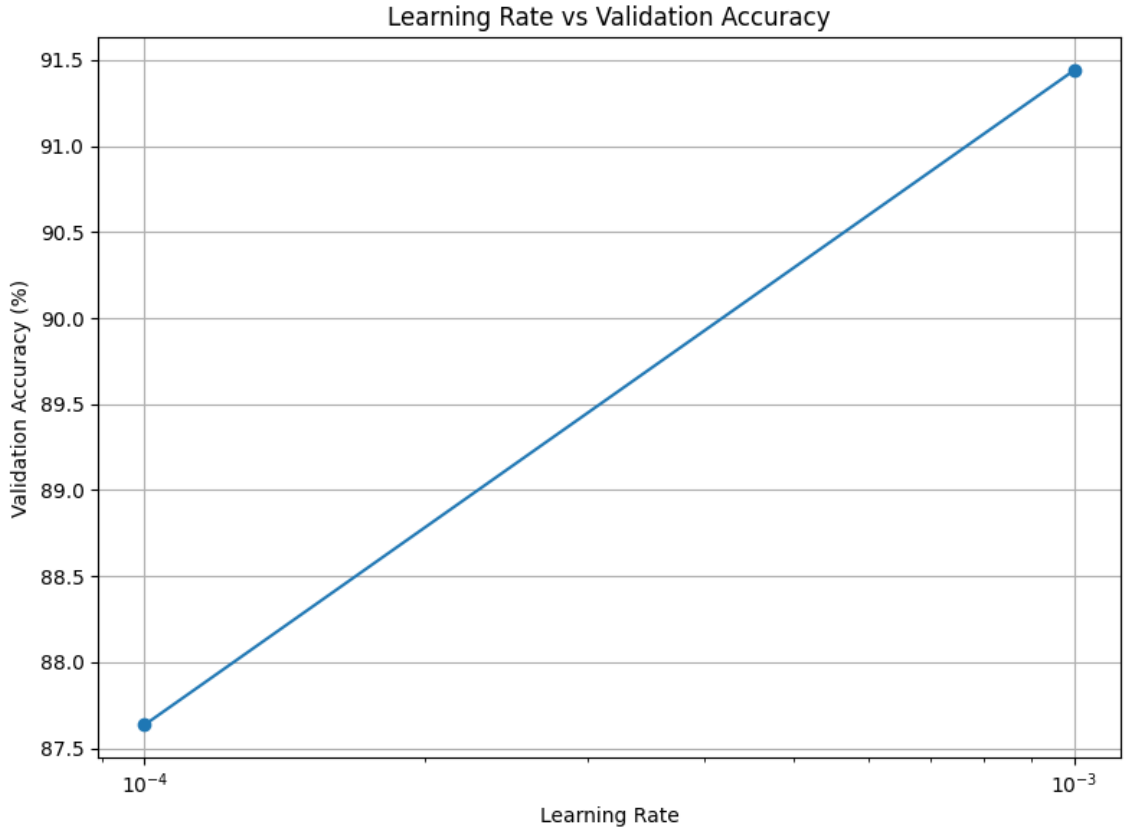
Adam and RMSProp showed a much faster reduction in training loss compared with SGD. SGD retained a high final loss of approximately 3.08, while Adam achieved the lowest loss of approximately 0.043. This demonstrates the advantage of adaptive optimization methods for faster convergence.

Plot 7: Validation Accuracy vs Epoch for Different Optimizers



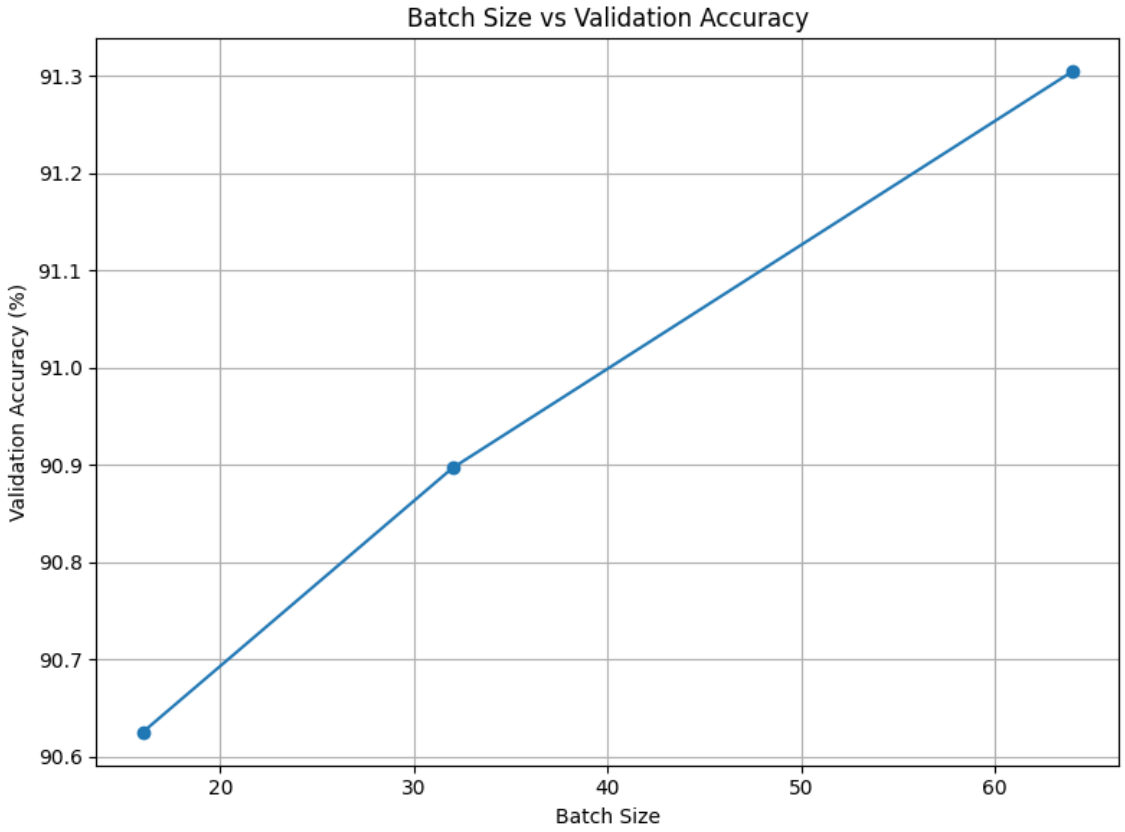
Validation accuracy increased significantly for Momentum, RMSProp, and Adam, whereas SGD achieved only approximately 23.37%. Adam obtained the best validation accuracy of approximately 90.49%. The results show that the choice of optimizer has a major influence on model performance.

Plot 8: Learning Rate vs Validation Accuracy



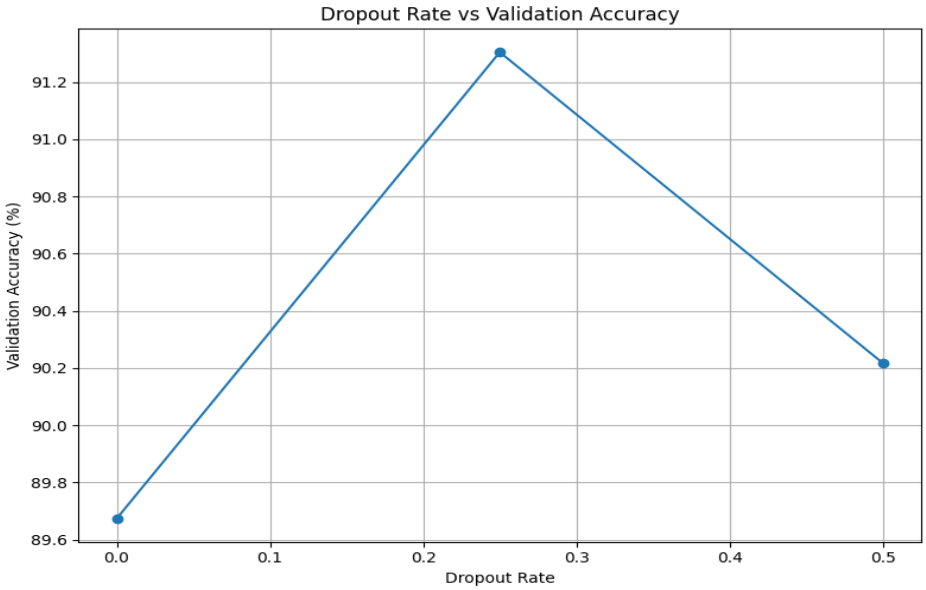
A learning rate of 0.001 achieved the best validation accuracy of approximately 91.44%, while 0.0001 achieved approximately 87.64%. The smaller learning rate resulted in slower convergence within the limited number of epochs. This demonstrates the importance of selecting an appropriate learning rate.

Plot 9: Batch Size vs Validation Accuracy



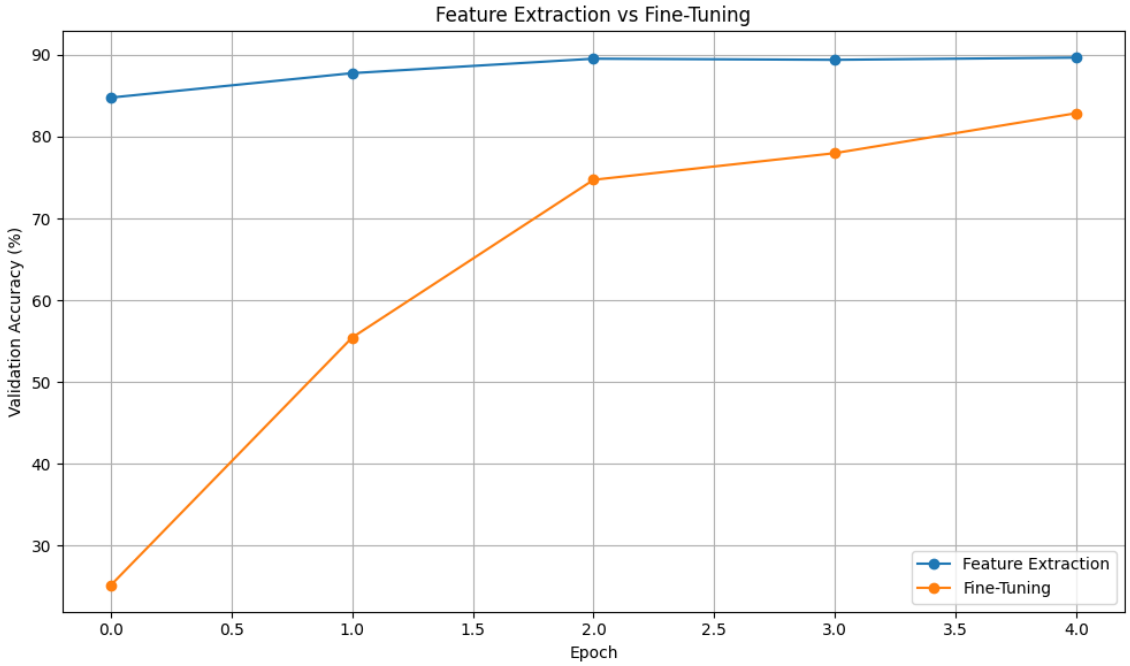
Validation accuracy increased slightly as the batch size increased from 16 to 64. Batch size 64 achieved the highest validation accuracy of approximately 91.30%. Larger batches provided more stable gradient estimates for this experiment.

Plot 10: Dropout Rate vs Validation Accuracy



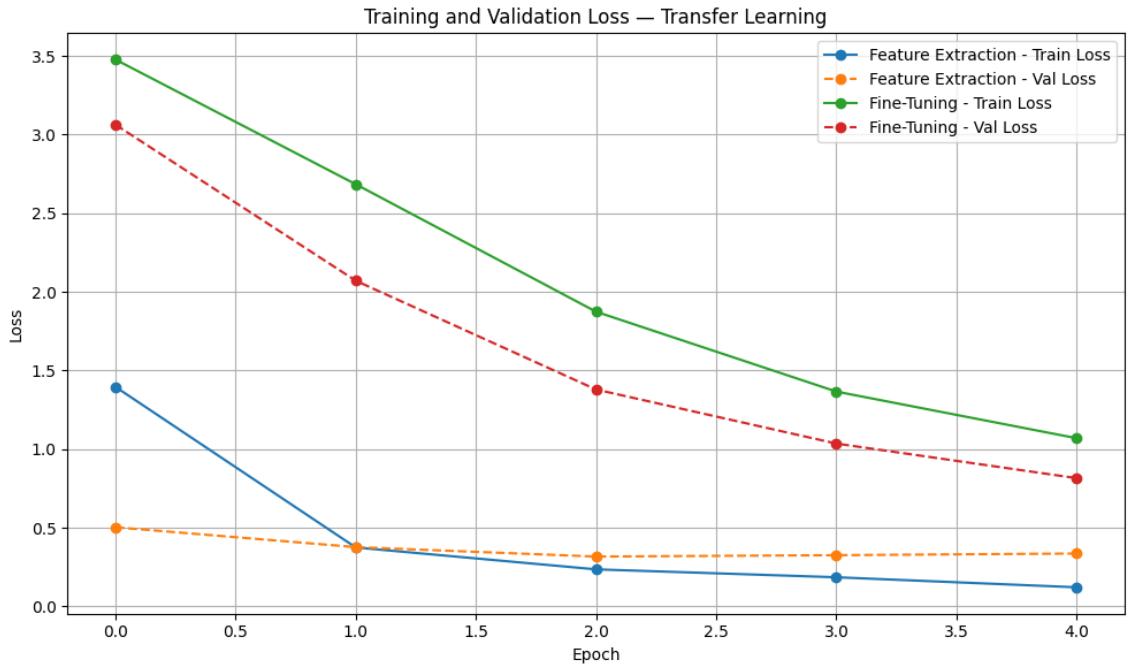
A dropout rate of 0.25 achieved the highest validation accuracy of approximately 91.30%. Without dropout, the model showed signs of overfitting, while a dropout rate of 0.50 reduced performance. Therefore, moderate dropout provided the best balance between regularization and learning capacity.

Plot 11: Feature Extraction vs Fine-Tuning



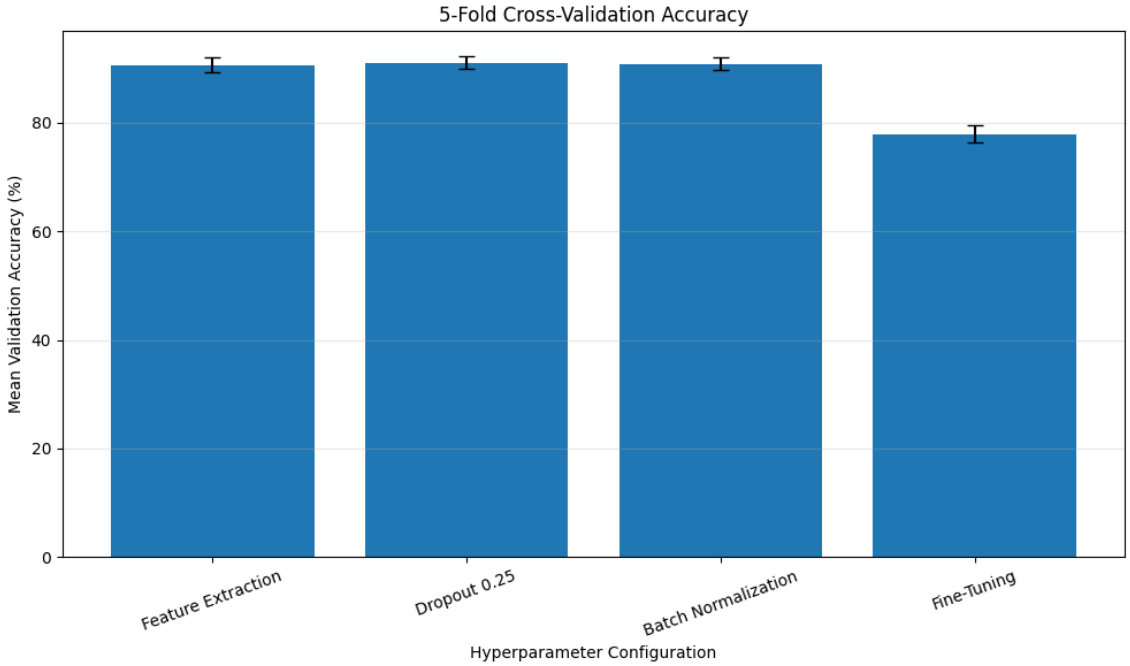
Feature extraction achieved higher validation accuracy during the experiment, whereas fine-tuning started with lower accuracy and gradually improved. Since fine-tuning was performed for only five epochs, the pretrained layers may not have had sufficient time to adapt fully. Fine-tuning generally requires careful learning-rate selection and sufficient training time.

Plot 12: Training and Validation Loss for Transfer Learning



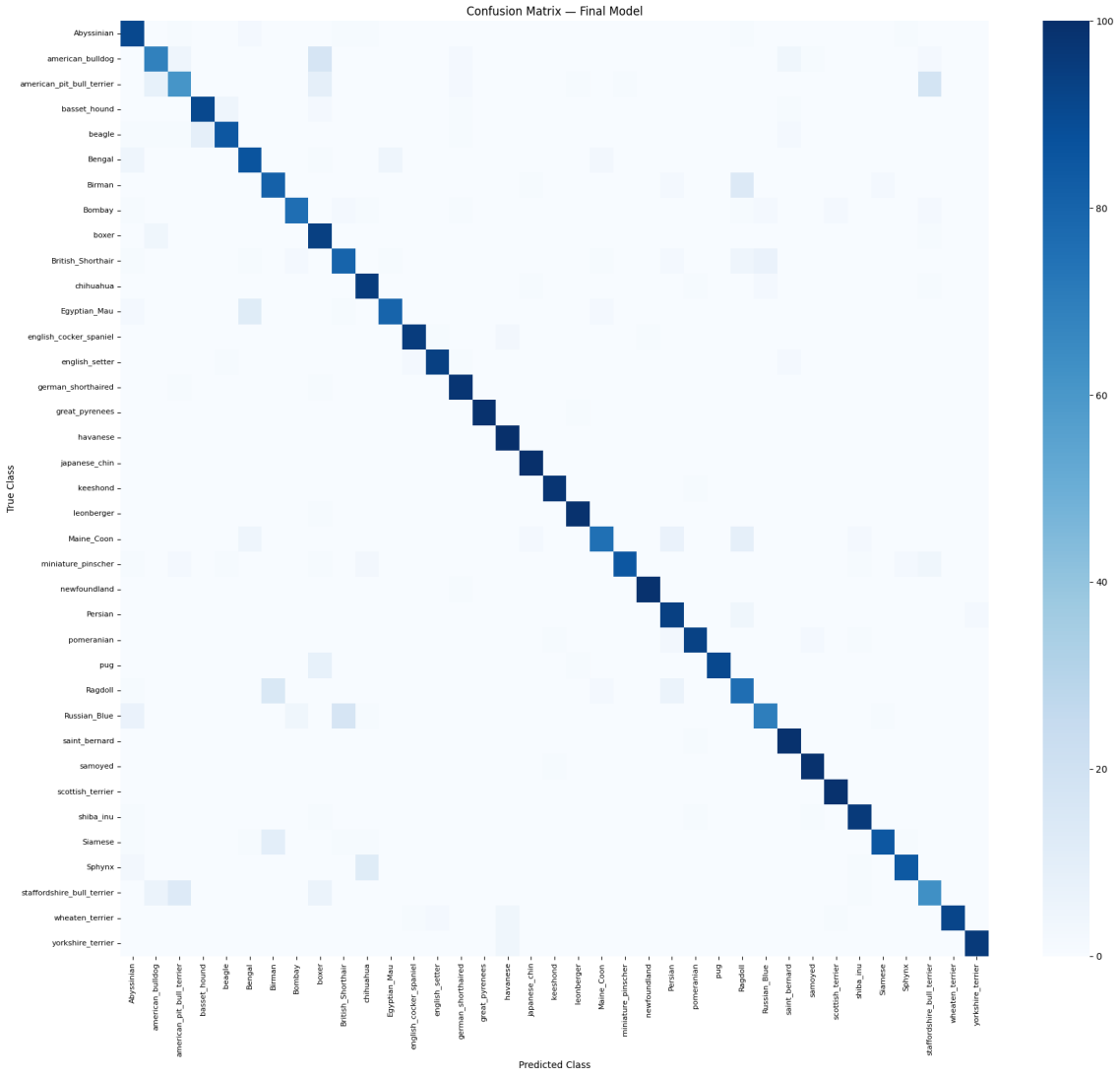
Both feature extraction and fine-tuning showed decreasing training and validation loss over the epochs. Fine-tuning started with a considerably higher loss but gradually reduced it during training. This behavior occurred because additional pretrained layers were made trainable and required adaptation to the new dataset.

Plot 13: 5-Fold Cross-Validation Accuracy



The plot compares the mean validation accuracy and standard deviation of four promising configurations. The Dropout 0.25 configuration achieved the best mean cross-validation accuracy of **91.03%** with a standard deviation of **1.20%**. This indicates good performance with relatively consistent results across different folds.

Plot 14: Confusion Matrix



The confusion matrix shows the class-wise prediction performance of the final model on the independent test set. Several classes, such as *great pyrenees*, *newfoundland*, and *scottish terrier*, were classified with very high accuracy, while visually similar breeds showed greater confusion. The final model achieved an overall test accuracy of **89.07%**.

15Source Code:

`https://github.com/PranavVarshin/DeepLearning/tree/main/Lab-5`

16. Expected Outcome

Students should experimentally demonstrate that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices. They should understand the MobileNetV2 architecture, perform transfer learning and fine-tuning, and select a reliable final configuration using 5-fold cross-validation.

17.Conclusion:

In this experiment, the effects of weight initialization, regularization, batch normalization, optimizers, learning rate, batch size, dropout, transfer learning, and fine-tuning were studied using the Oxford-IIIT Pet dataset. Different configurations were evaluated to understand their impact on model convergence and classification performance.

Among the tested configurations, the Dropout 0.25 model achieved the best mean cross-validation accuracy of **91.03%** with a standard deviation of **1.20%**. The final model achieved a test accuracy of **89.07%**, with precision, recall, and F1-score close to **89%**. The experiment demonstrated that proper selection of initialization, regularization techniques, optimizers, and hyperparameters plays an important role in improving the performance and generalization of deep learning models.

18. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: `https://www.tensorflow.org`
6. Keras Documentation: `https://keras.io`

- NAME: T Pranav Varshin(24110413)
- Reg No: 24011101116
- Section: AIDS-B